

Model-Driven Software Development

Programming eXperience



Miguel Campusano

SDU Software Engineering

Previous class

Metamodel & Functions

Summary

Metamodel programming

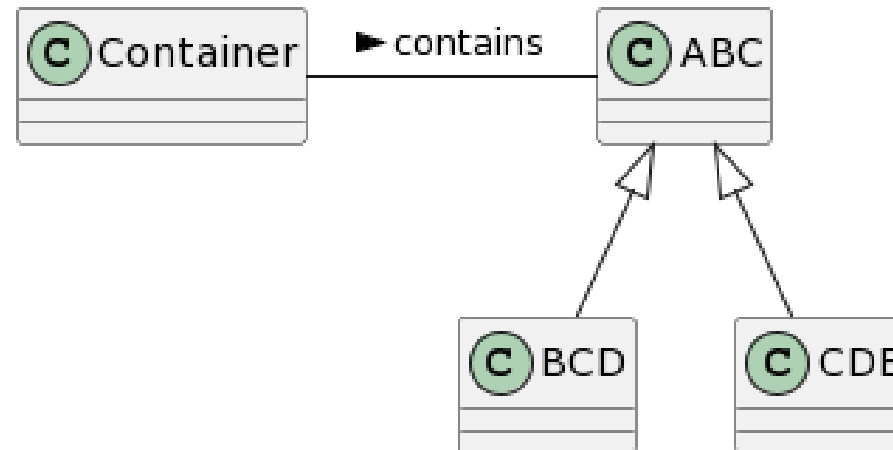
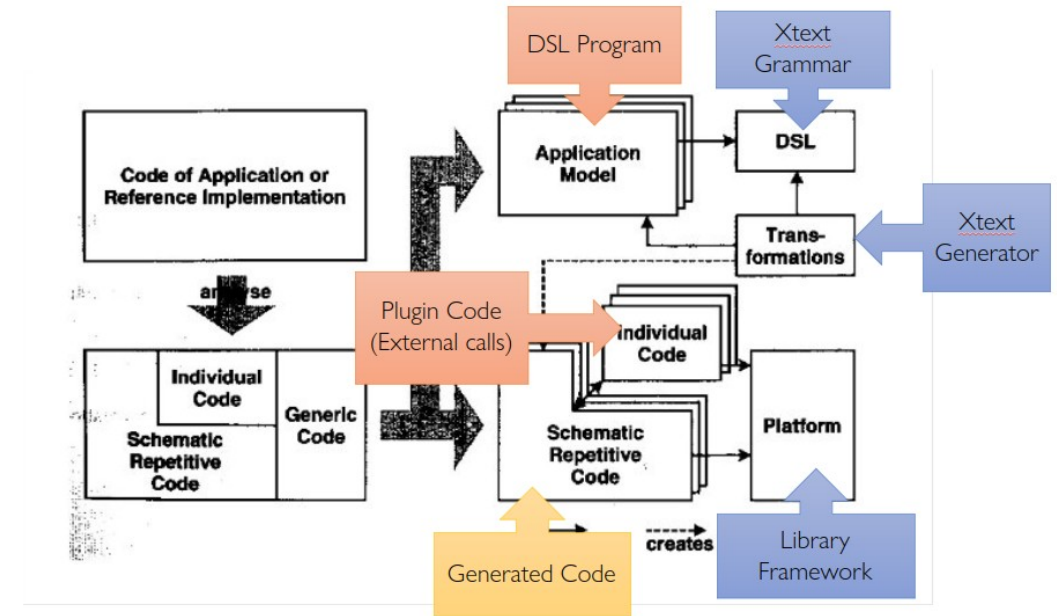
External functions

Project: Questions & Feedback

Next?

Programming eXperience

Type Systems



Metamodel Programming

Xtext infers the metamodel from the grammar specification

For each rule, a class is created*

Alternatives (`ClassA` | `ClassB`) generates subtyping (subclasses)

Assignment generates instance variables

Use `returns` keyword to change class name

Actions ({ `ClassName` }) to create hierarchy

Tree rewrite action (from lec06)

Cross-reference ([`ClassName`])

Cross-reference over two rules?

Class Description V4: Complex functions

```
system University
external legalCPR(String) : Boolean
external complexExp(Number,Number) : Number
```

```
entity Person
  attribute name is String
  attribute age is Number
  attribute cpr is String
  require age >= 0
  require legalCPR(cpr)
```

...

```
require 0 <= complexExp(10,attr)
```

S ::= 'system' ID X* E* R*

X ::= 'external' ID '('(Type (',' Type)+)?')' : T

E ::= 'entity' ID (At|Q)*

At ::= ('attribute' | '-') ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

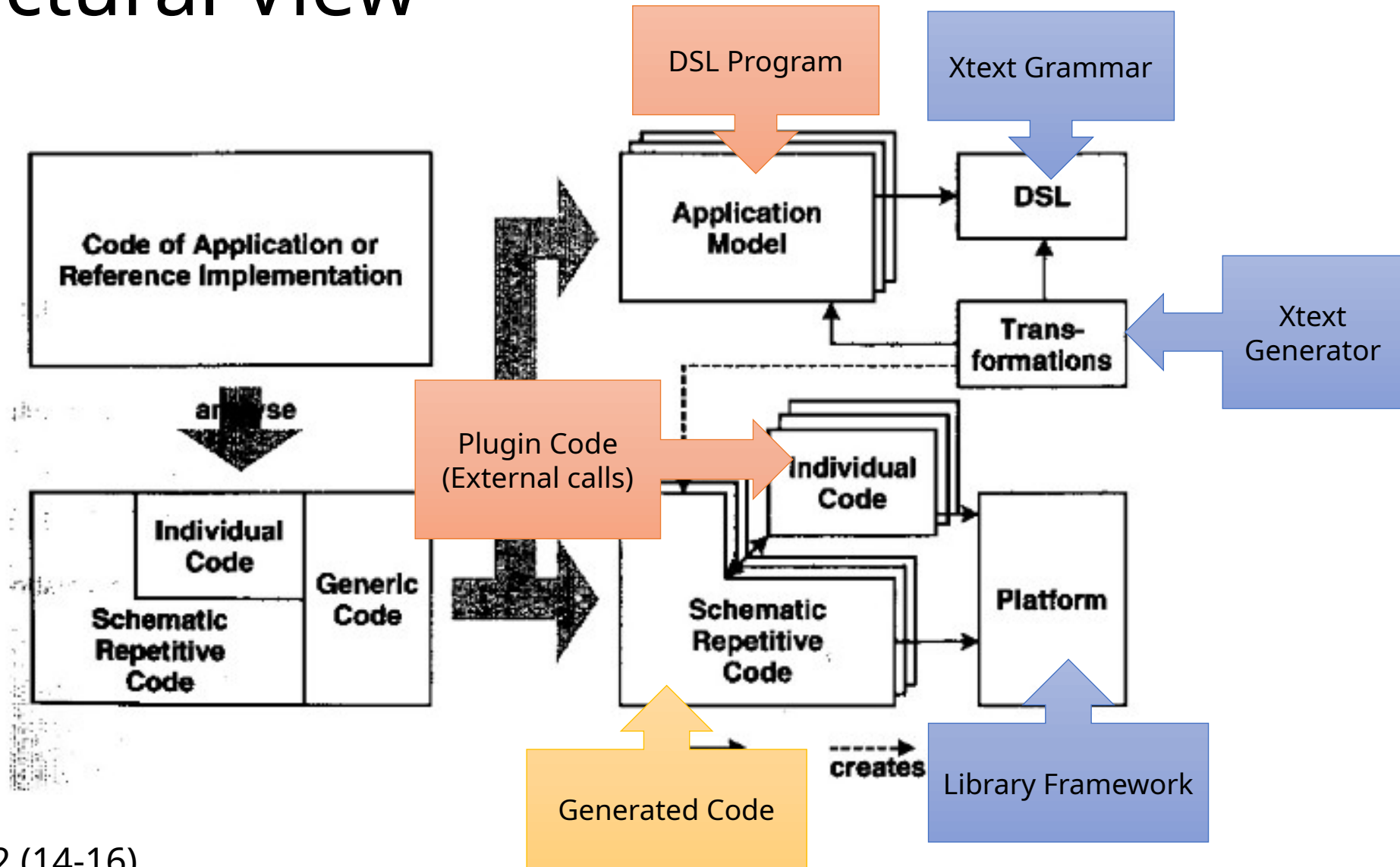
I ::= ID_E ('is' | '->') ID_E

C ::= ID_E ('1-1' | '*-*') ID_E

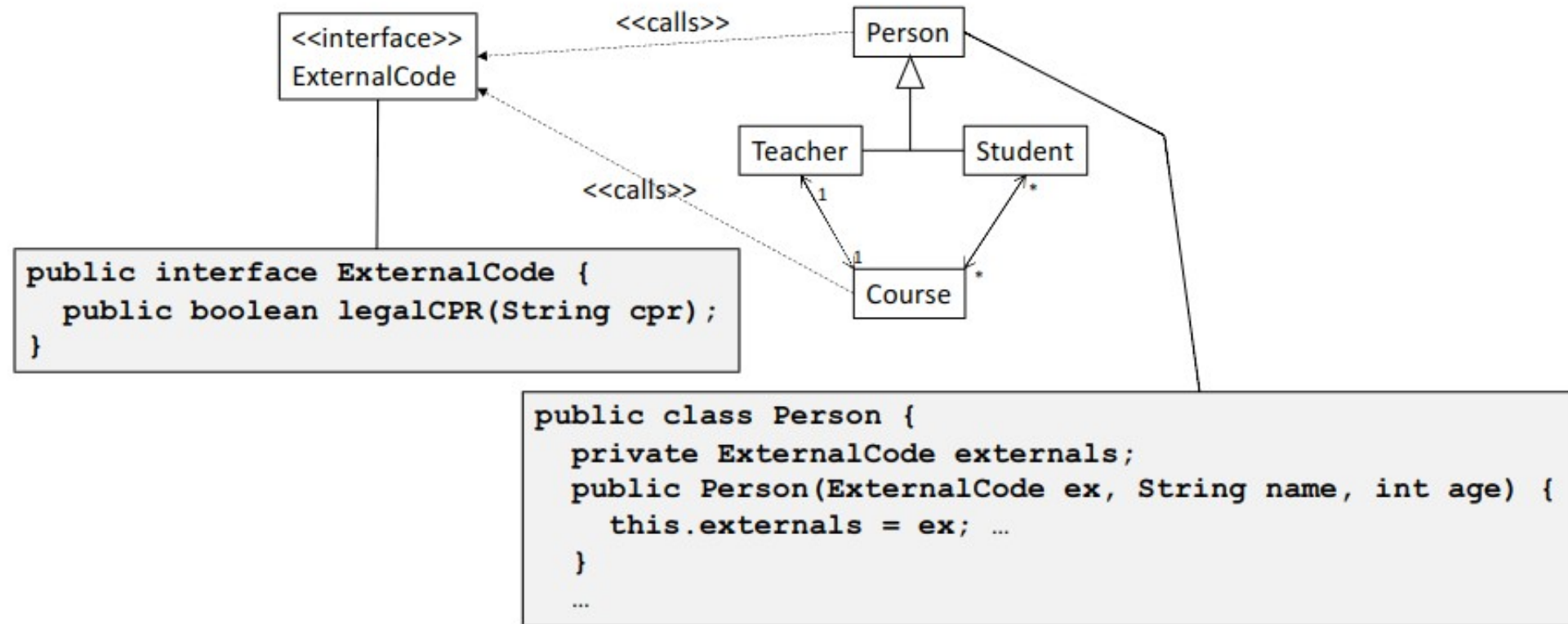
Exp ::= ... | Xc

Xc ::= ID^x '('(Exp (, Exp)+)?')'

Architectural View



External calls: Interface



Model-Driven Software Development

Programming eXperience



Miguel Campusano

SDU Software Engineering

Today

Programming eXperience (PX)

Live Programming

How to: Traceability

Visual DSLs

Programming eXperience

Programming Experience

*Do programmers create text that is transformed into **running behavior** (the old way), or do they operate on **behavior directly** ("liveness"); are they exploring the **live domain** to understand the true nature of the requirements; are they like authors creating **new worlds**; **does visualization matter**; is the experience **immediate, immersive, vivid and continuous**; do **fluency, literacy, and learning** matter; do they **build tools, meta-tools**; are they creating **languages** to express **new concepts quickly and easily**; and curiously, is **joy** relevant to the experience?*

- Programming Experience Workshop

Programming Experience

User eXperience (UX) applied for programming (PX)

UX: users interaction with systems

PX: *programmer* interaction with programming systems

Programmer: developer, coder, engineer, user, ...

Your Programming Experience

We have experienced applications with UX problems ...

What about...

Your experience with (any) programming environments

Good: start with G

Bad: start with B

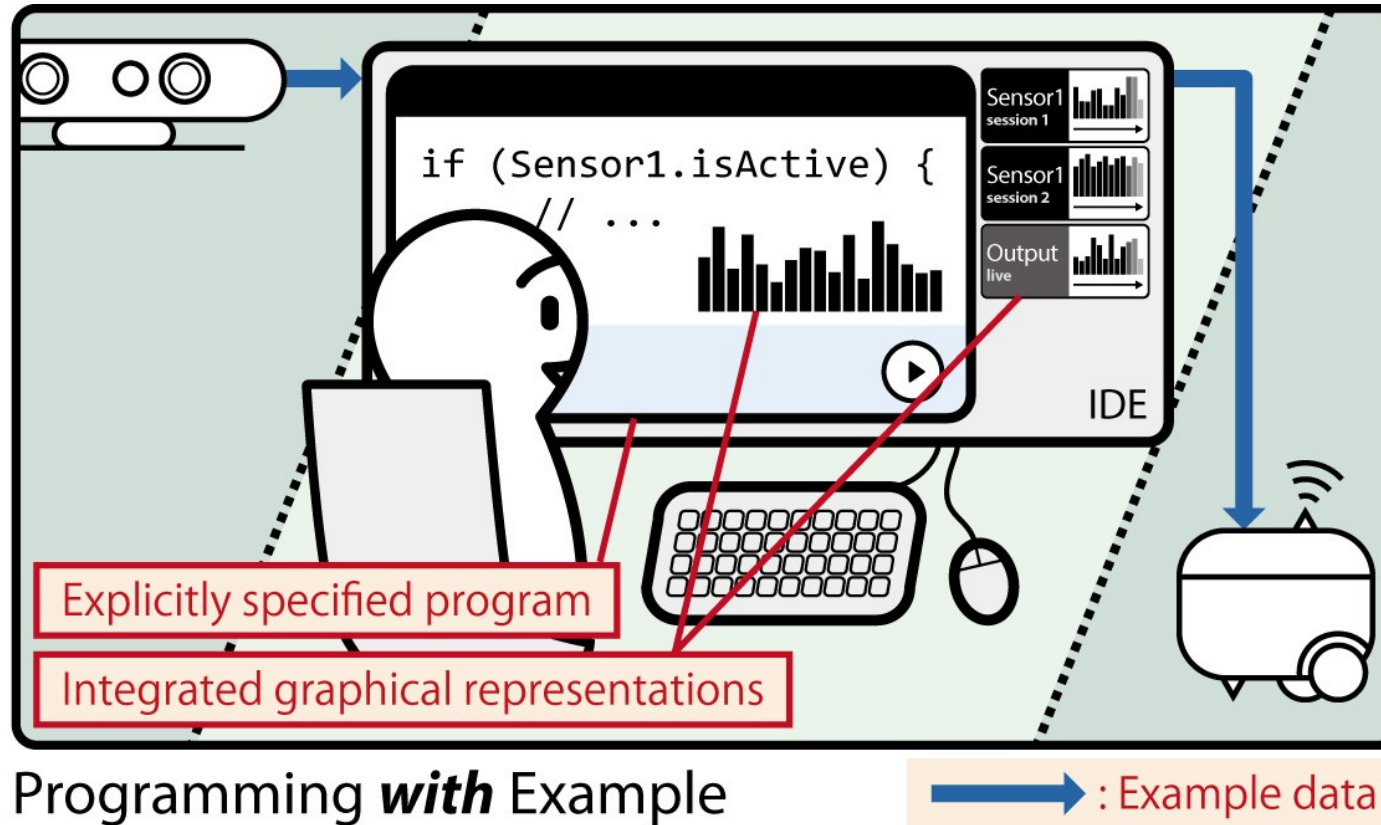
Comments: start with C

[PollEv.com/mica](https://pollev.com/mica)

Your Programming Experience

Is it really important?

PX example: Program with Example



What aspect do we want to improve?

Improving:

Usability

Learnability

Comprehension

Efficiency of coding

Enjoyment

...

For whom?

Coders / Developers

Designers

Architects

Engineers

Machine Operators

Artists

Citizens (low-code)

...

Development practices

Coding

Debugging

Refactoring

Design

Program Comprehension

...

What aspect do **I** want to improve?

Improving:

Usability

Learnability

Comprehension

Efficiency of coding

Enjoyment

...

For whom?

Coders / Developers

Designers

Architects

Engineers

Machine Operators

Artists

Citizens (low-code)

...

Development practices

Coding

Debugging

Refactoring

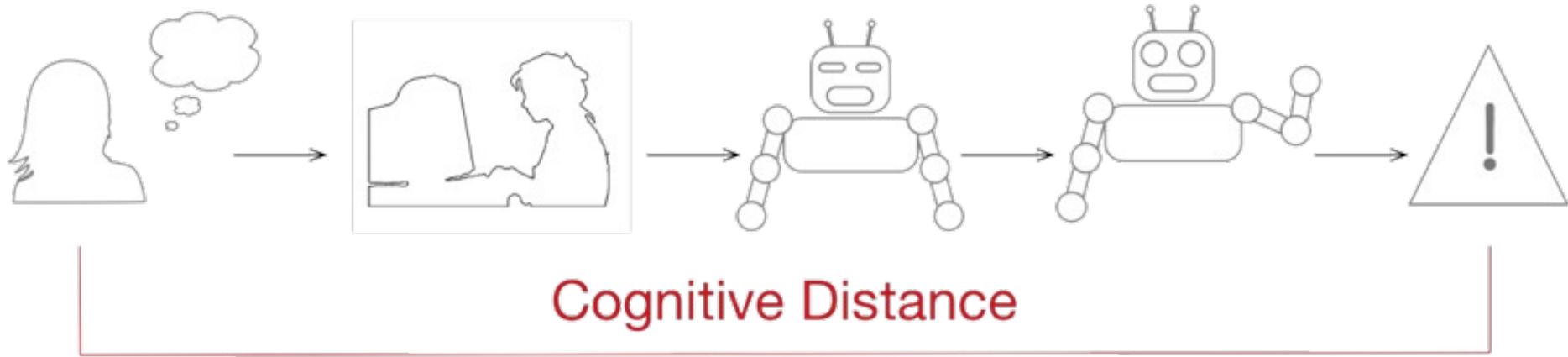
Design

Program Comprehension

...

Meaningful Feedback

Cognitive Distance



Mental Model => ... => Program execution

Meaningful Feedback

The program gives proper feedback to developers

Developers can reason and take actions

Logs are not proper feedback

Meaningful Feedback

Rule of thumb: Mental Models / Metamodel

State machine → States and Transitions

Interactive Fiction → Graph

Class Description → Class Diagram

...

However, it may not be enough

Mental model may not be clear enough → Interactive Fiction, Graph, and Player

Metamodel heavily dependent on domain (it is context specific)

What about feedback from programs in GPL?

There may be more information developers need to understand execution

Meaningful Feedback: State machines

[illegible]

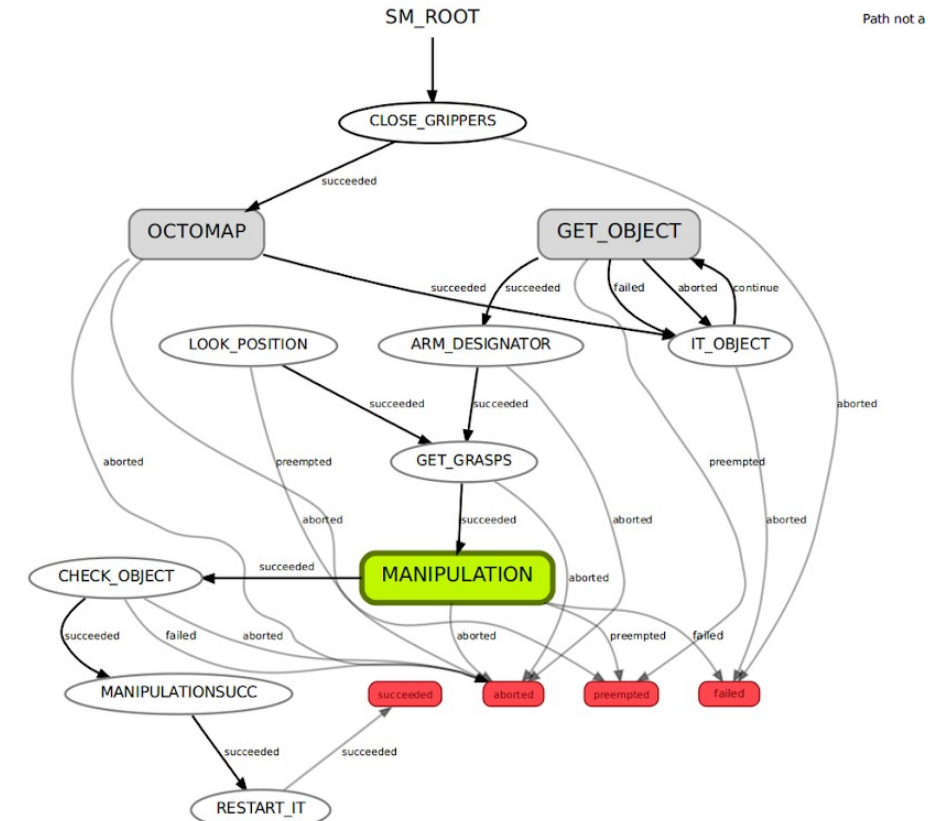
Runtime information

Runtime state

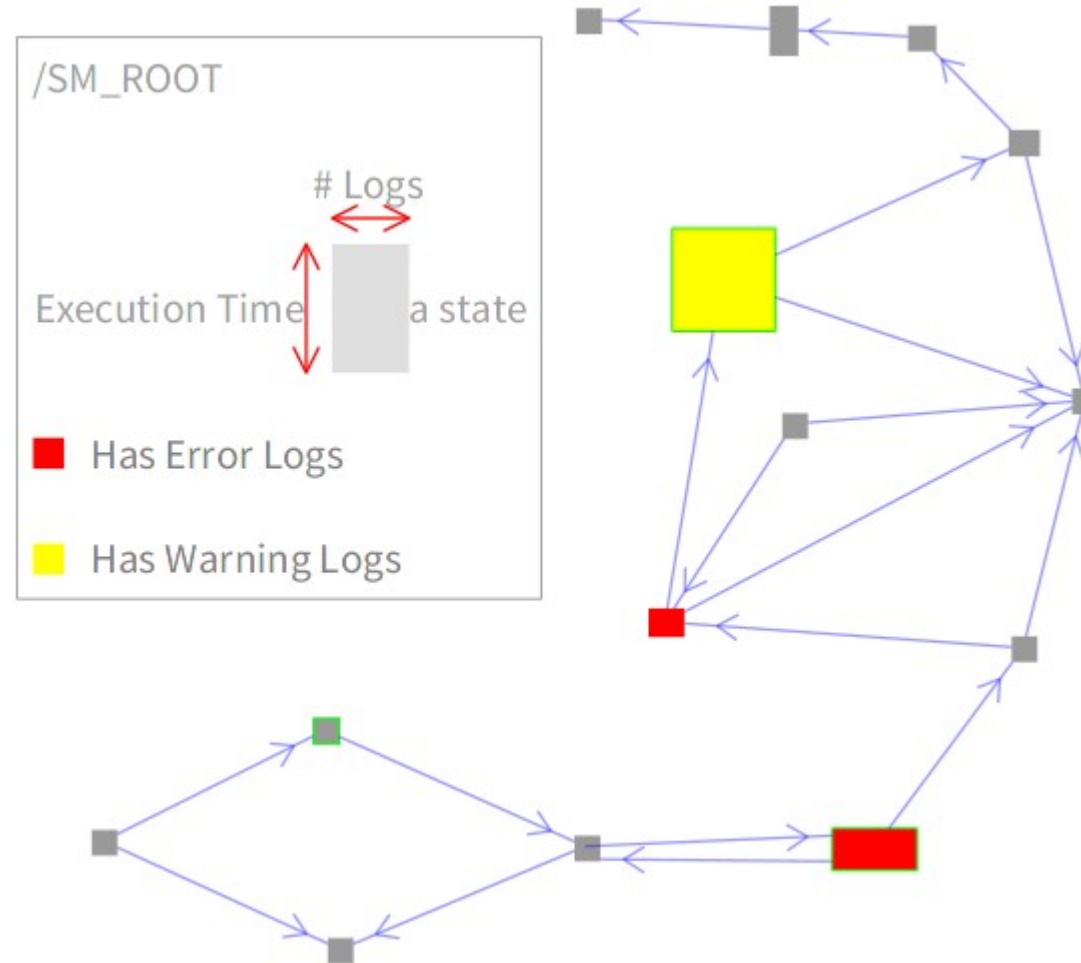
Logs

Metamodel

State machines



Meaningful Feedback: state machines



Meaningful Feedback: cobots

Rule of thumb: Mental Models / Metamodel

Metamodel? Still, specific domain

However, it may not be enough

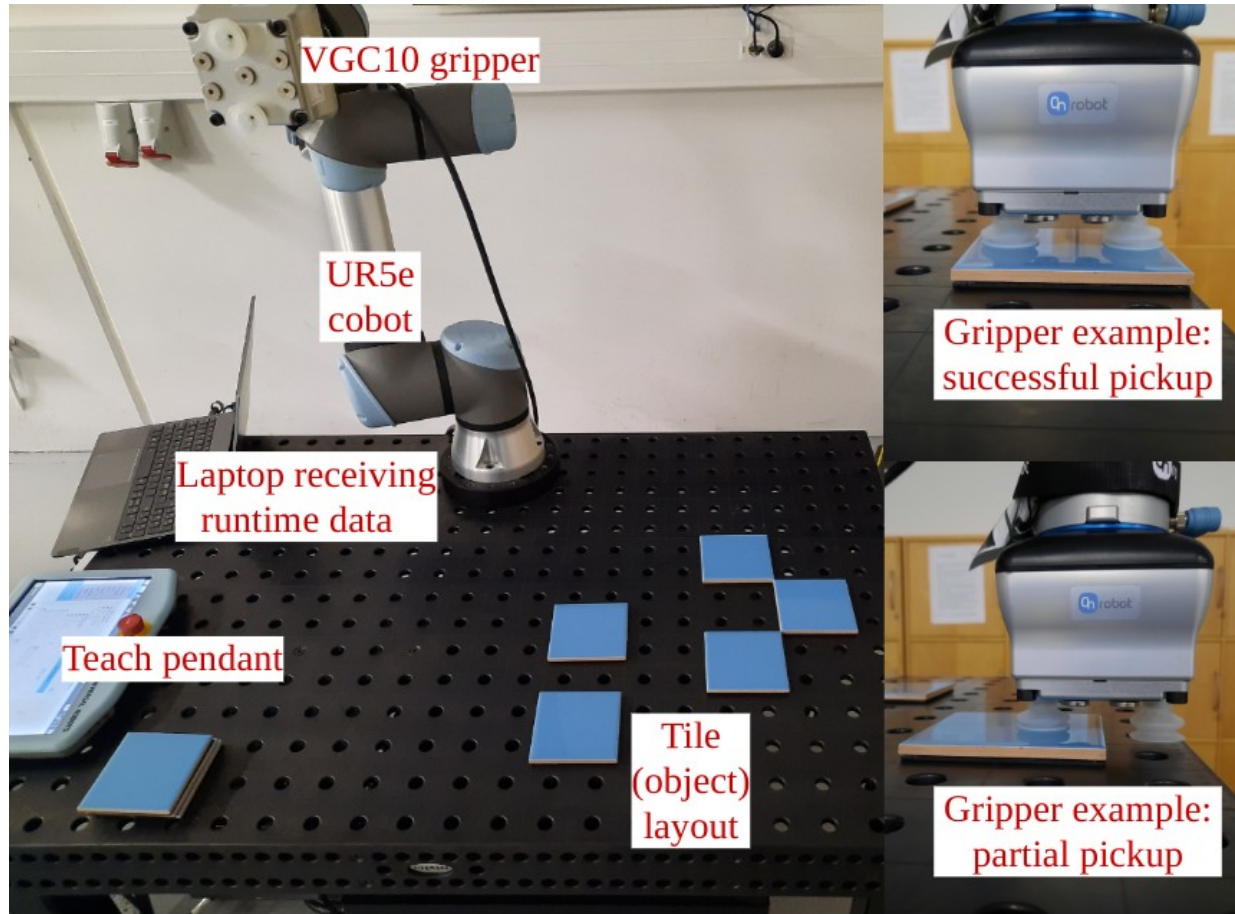
Mental model may not be clear enough

~~Metamodel heavily dependent on domain (it is context specific)~~

~~What about programs in GPL?~~

There may be more information developers need to understand execution

Meaningful Feedback: cobots



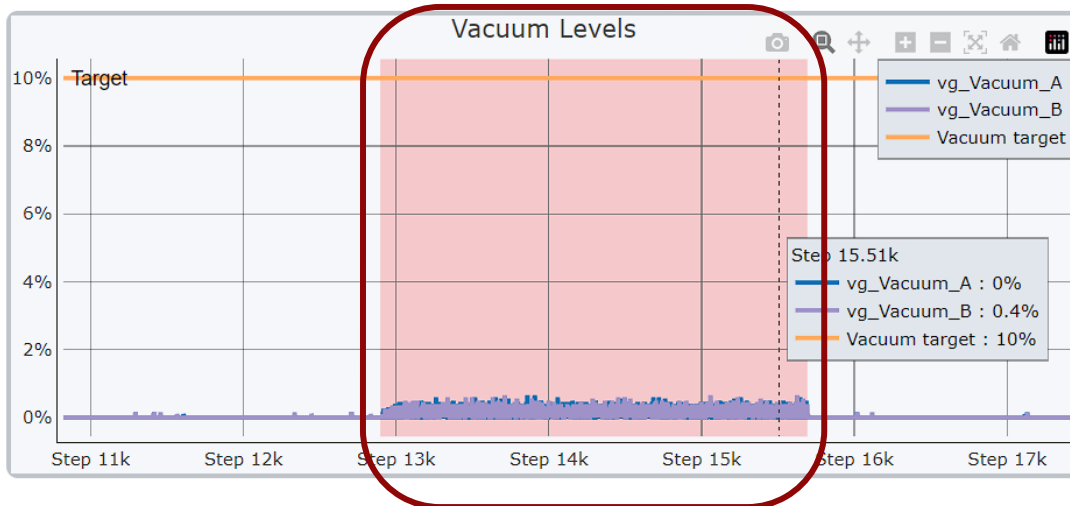
Runtime information

Program runtime state

Robot runtime state

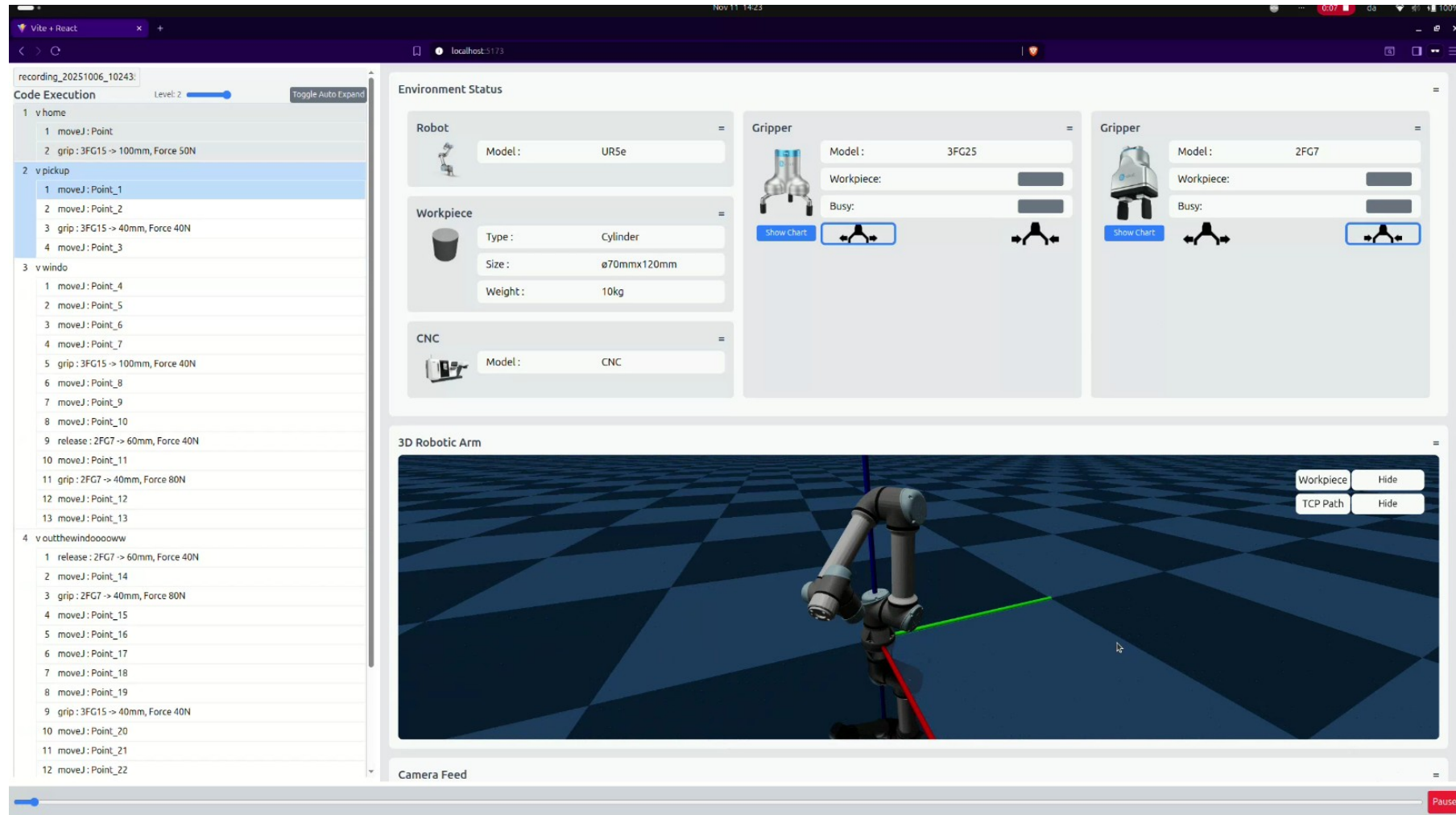
Meaningful Feedback: cobots

Visualization with Source code
Highlight *current* execution line



```
script
136  vg10_grip(2, vacuum_level, 0, False)
137
138  # Lift up to stand clear
139  pickup_pose[2] = travel_height
140  movel(pickup_pose, a=move_acceleration, v=move_speed)
141
142  # Move to the final/dropoff position (Should be movej, but because of qnear shenanigan
143  movel(dropoff_pose, r=0.1)
144
145  dropoff_lowered = dropoff_pose
146  dropoff_lowered[2] = dropoff[2]
147  # Move straight down to drop off the item
148  movel(dropoff_lowered)
149
150  # Open the tool
151  #popup("Drop the vacuum here!", "Dropping off the tile", False, False, True)
152
153  # channel, timeout, autoidle, toolindex
154  # Channel: 0 = A, 1 = B, 2 = Both
155  # timeout is how long the command waits for the vacuum to drop, before giving an error
156  # it is given in seconds : 0.6 = 600 ms
157  # autoidle :
158  # Whether the release valve should be automatically turned off when the release is
159  # and the robot has moved 5 cm away from the release position
160  # tool index is only applicable if more than one gripper is mounted
161  vg10_release(2, 0.6)
```

Meaningful Feedback: cobots



FERAViz – coming soon...

Live Programming

What is Live Programming?

For you...

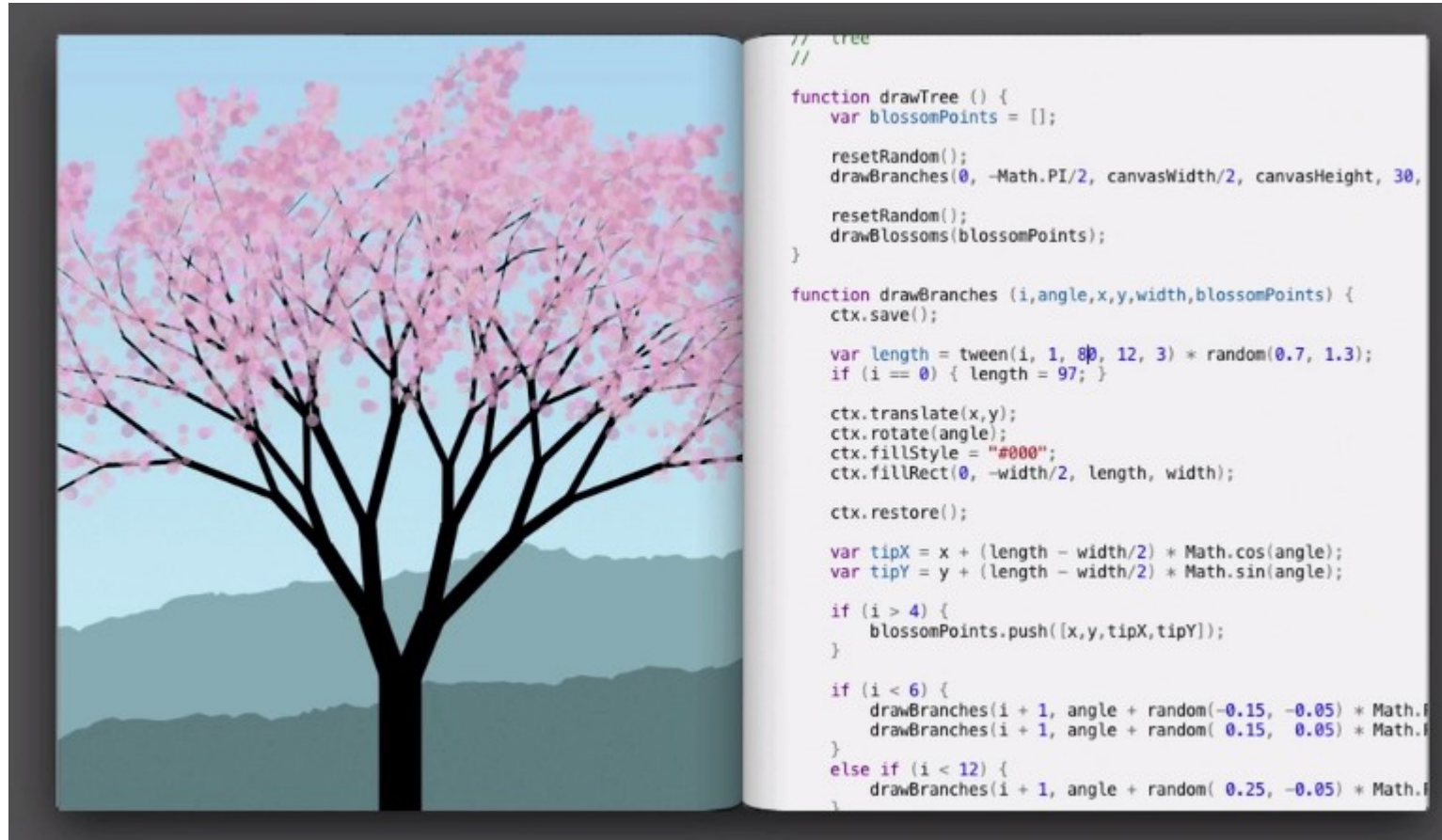
What is live programming?

What do you understand of live programming?

What is *liveness*?

...

Live Programming example



Bret Victor – Inventing on Principle (CUSEC '12) - <https://youtu.be/PUv66718DII>

Live Programming

Combination of 2 concepts

Immediate Connection

Changes are reflected immediately

Meaningful Feedback

The program gives proper feedback to developers

Logs are not proper feedback

Level of Liveness

Level 1: Informative

Visual representation

Level 2: Informative and significant

Executable visual representation

Level 3: Informative, significant and responsive

Incomplete liveness: update programs, but using devs actions (buttons)

Level 4: Informative, significant, responsive and live

Liveness

Steven L. Tanimoto – VIVA: a visual language for image processing – Journal of Visual Languages & Computing (1990)

Level of Liveness: more

Level 5: Tactically predictive

Program itself plans ahead, predicting next programmers' actions

Level 6: Strategically predictive

Program can create itself, via developers intention

Steven L. Tanimoto – A perspective on the evolution of live programming – LIVE'13

Level of Liveness

Level 1: Informative

Visual representation

Level 2: Informative and significant

Executable visual representation

Level 3: Informative, significant and responsive

Incomplete liveness: update programs, but using devs actions (buttons)

Level 4: Informative, significant, responsive and live
Liveness

Level 4 is not enough! We need *proper* feedback...

Live Programming

No live feedback

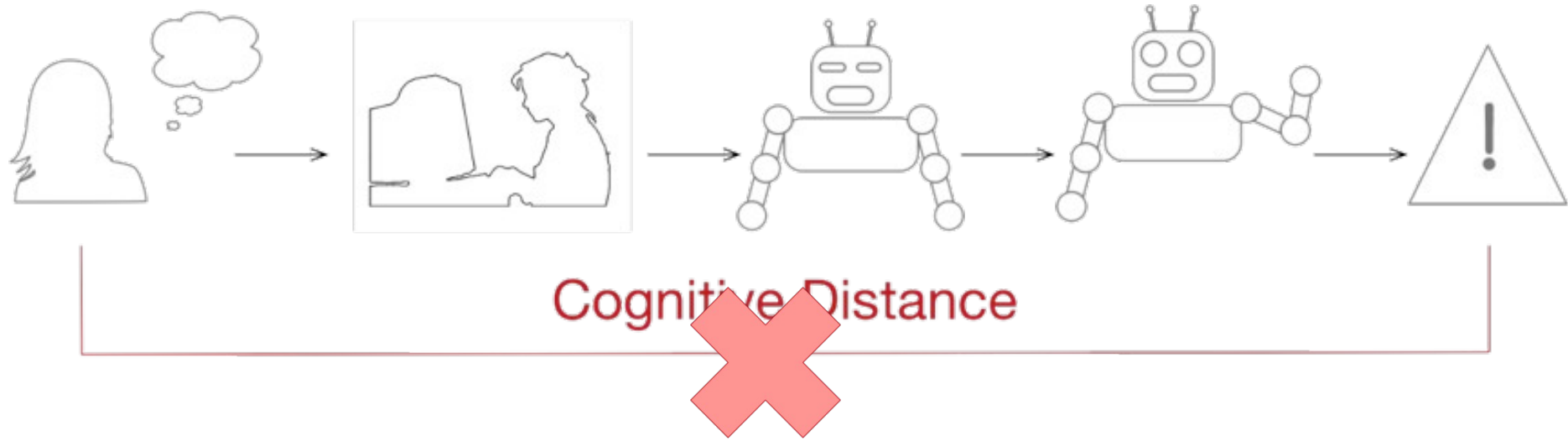


Live feedback



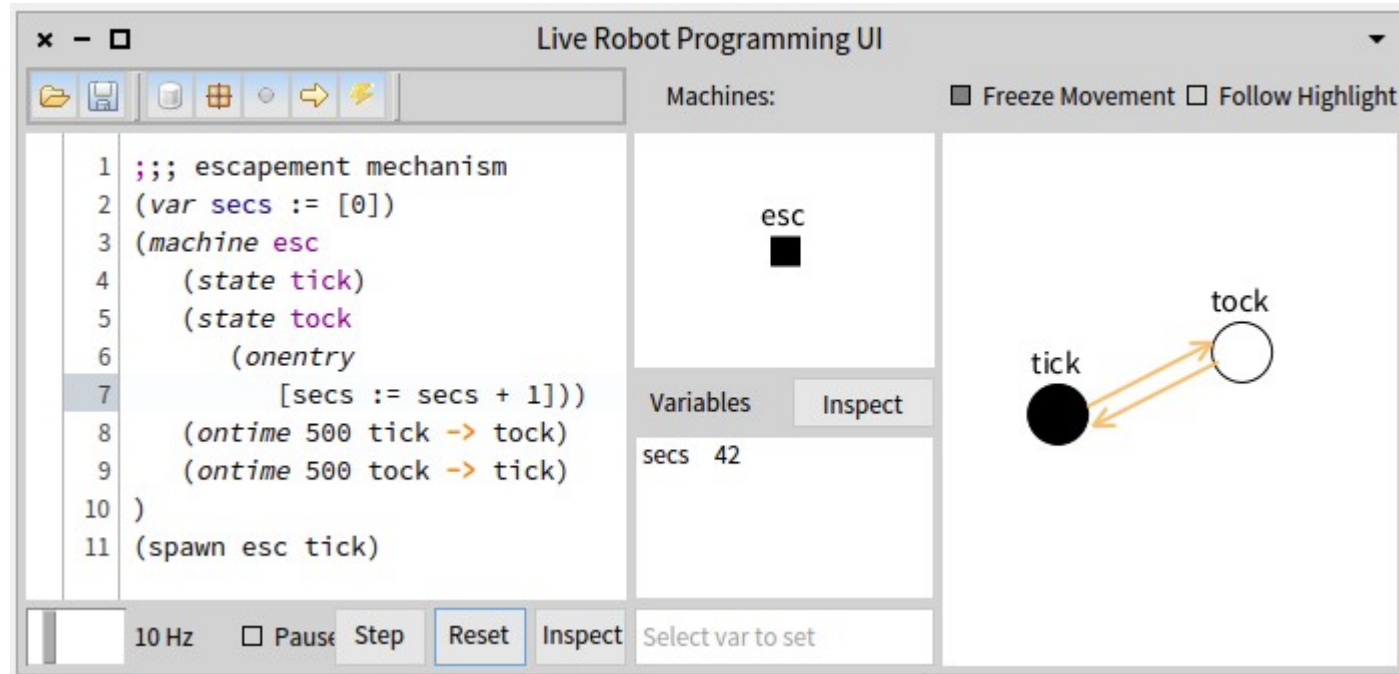
Hancock, Christopher Michael - Real-time programming and the big ideas of computational literacy.

Cognitive Distance



Mental Model $\Leftrightarrow$ Program execution

Live Programming in robots



Live Programming in drones



Carolina Hernández – Bsc: LRP and AR.Drone (UChile 16) - <https://youtu.be/ENmuGU4gBh4>

How to: LRP

Mental models / Metamodel: State machines

Meaningful Feedback:

- State machine visualizations

- Use mental model for feedback

Runtime information

- Trace runtime to source code

- Static Metamodel $\Leftrightarrow$ runtime state: variables + current state

How to: LRP

How to update a LRP program while it runs?

Program P is modified => new program P'

Program P is paused and *unloaded*

Program P' is loaded

Runtime state P is transferred to runtime state of P'

P' is executed

What happened if current runtime cannot be transferred?

Execution is restarted

Traceability

Model-aware vs model-ignorant generation

Model-aware

```
State a = new State()
a.onExecute{//lamda with code}
State b = new State()
...
t = new Transition(a, b, 'event');
...
Machine m = new Machine();
m.addState(...)...
m.addTransition(...)...
m.execute()
```

Model-ignorant

```
public void processEvent(String event){
    switch(currentState){ //on state
        case 0: //if in state 0
            switch(event) { //events
                case 'event': {
                    //process code
                    //next state if applicable
                }
            }

        case 1: //if in state 1 ...
    }
}
```

Traceability

- Trace Logging

- Tracking of runtime data

 - Events

 - Variable values

 - Line of Code Executed

 - ...

- Track back to code / mental model

Traceability in Xtext

Case 1: runtime visualization

Generated code and generated visualization

Connect code execution with visualization

Model-aware generation

Case 2: source code

What about DSL source code?

Traceability in Xtext: source code

What about DSL source code?

Source code → Generated code

Generated code → Source code

Traceability in Xtext: source code

What about DSL source code?

Source code → Generated code

Generate with LoC

Generate with Region

Generate with ...

```
import org.eclipse.xtext.nodemodel.util.NodeModelUtils
...

package «system.name»;
/*
line: «NodeModelUtils.findActualNodeFor(entity).getStartLine()»
region: «NodeModelUtils.findActualNodeFor(entity).getTextRegion()»
*/
public class «entity.name» {
```

Traceability in Xtext: source code

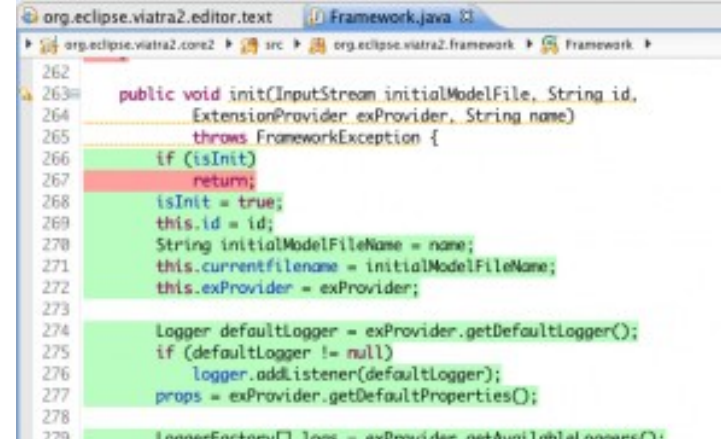
What about DSL source code?

Generated code → Source code

Eclipse plugin

Connect plugin with runtime info

Runtime server...



```
org.eclipse.viatra2.editor.text Framework.java
org.eclipse.viatra2.core2 src org.eclipse.viatra2.framework Framework
262
263 public void init(InputStream initialModelFile, String id,
264                 ExtensionProvider exProvider, String name)
265                 throws FrameworkException {
266     if (isInit)
267         return;
268     isInit = true;
269     this.id = id;
270     String initialModelFileName = name;
271     this.currentfilename = initialModelFileName;
272     this.exProvider = exProvider;
273
274     Logger defaultLogger = exProvider.getDefaultLogger();
275     if (defaultLogger != null)
276         logger.addListener(defaultLogger);
277     props = exProvider.getDefaultProperties();
278
279     LoggerFramework[] loggers = exProvider.getDefaultInitialLoggers();
```

Visual DSLs

What are Visual DSLs

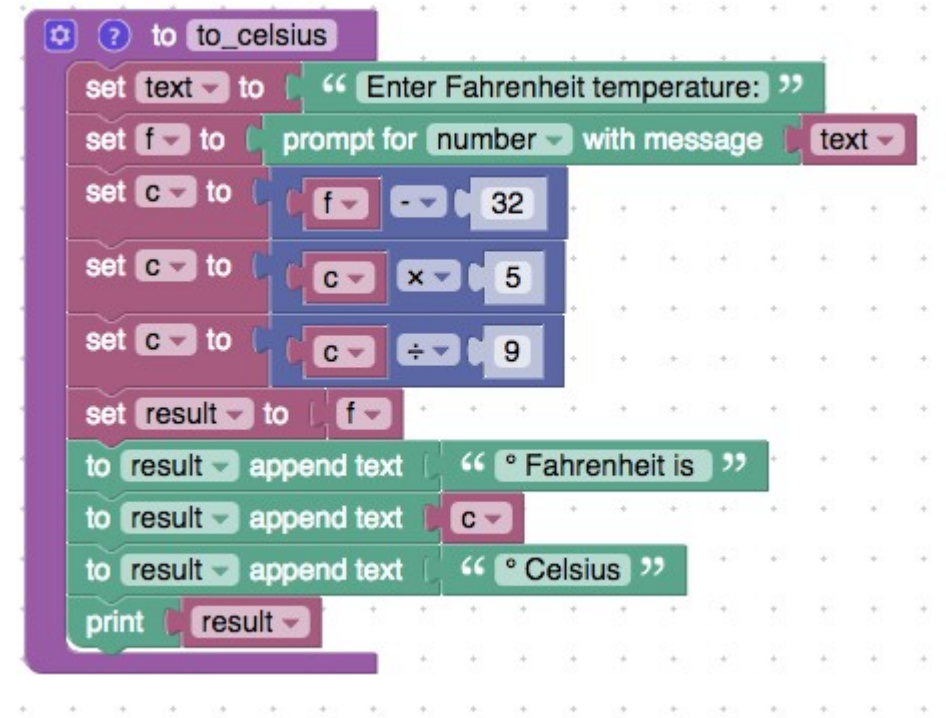
What do you understand about Visual DSLs?

Blockly

Visual DSL, Block-based programming language

Visual blocks that connects to each other

You can create our own definitions



Blockly

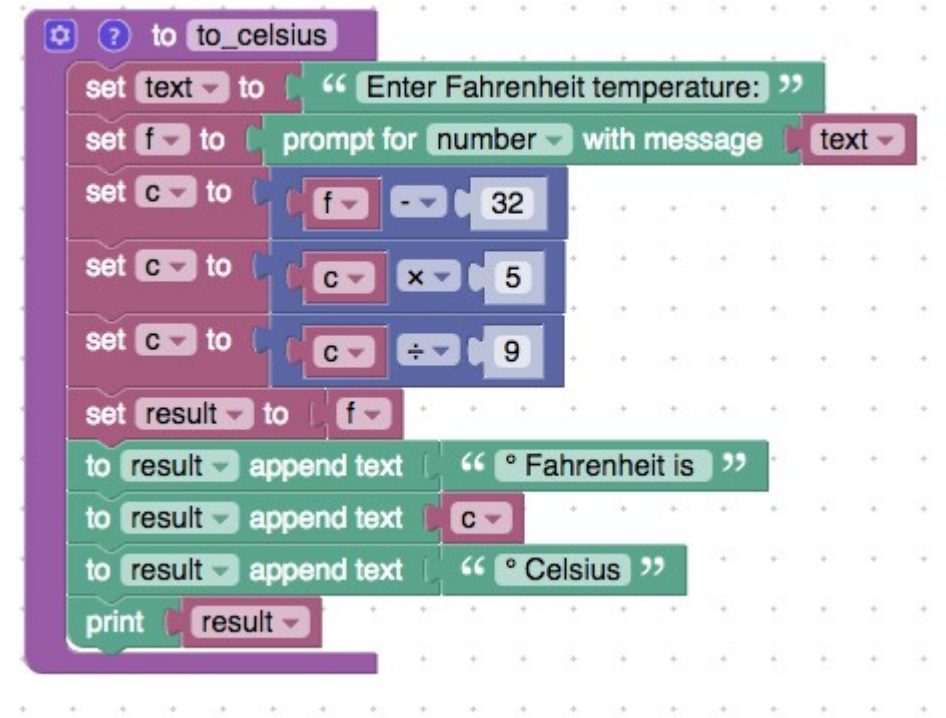
Grammar defined by blocks

Metamodel $\rightarrow$ Blocks $\rightarrow$ ~EBNF

AST

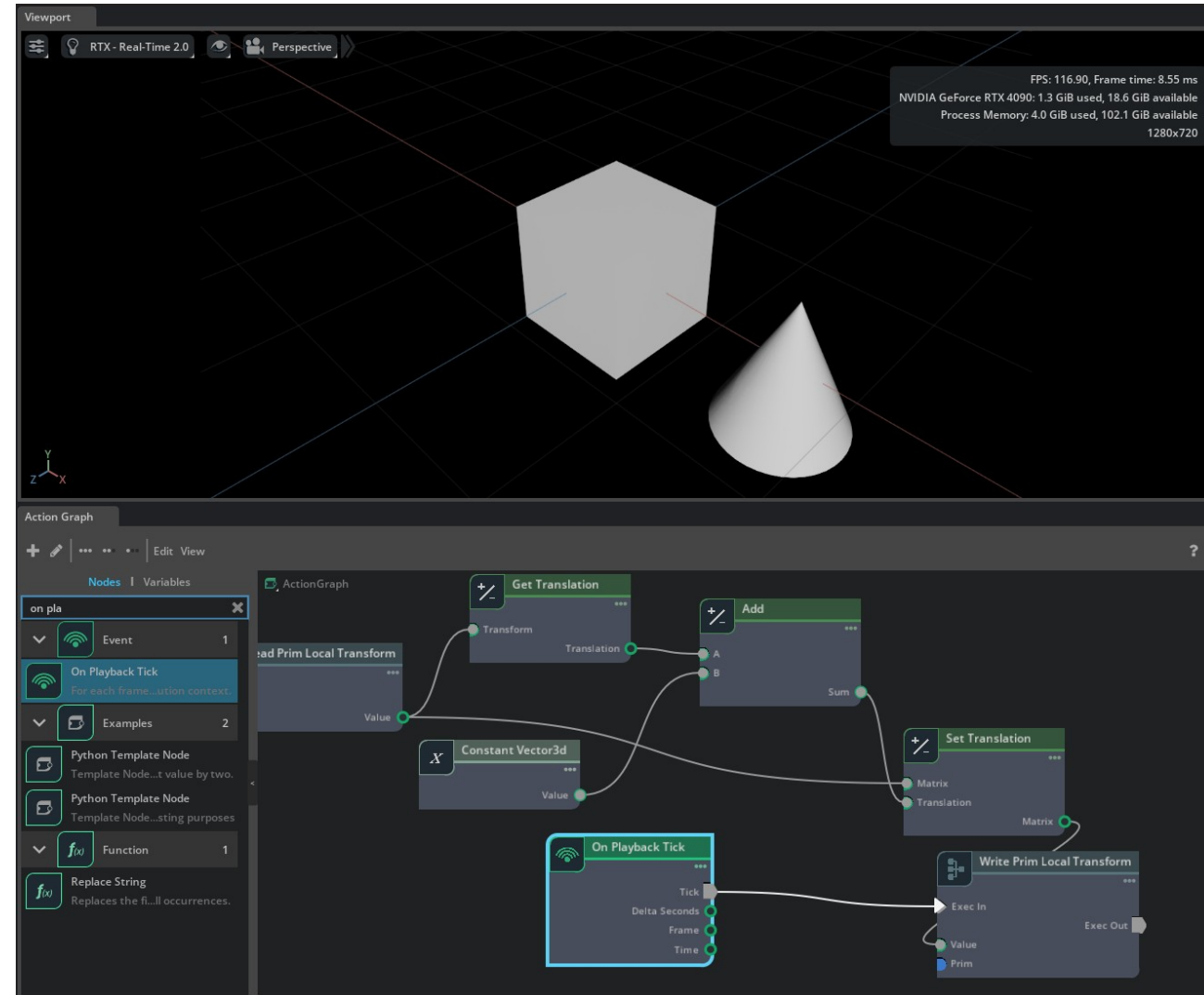
Code generator per block

Each block “generates” code



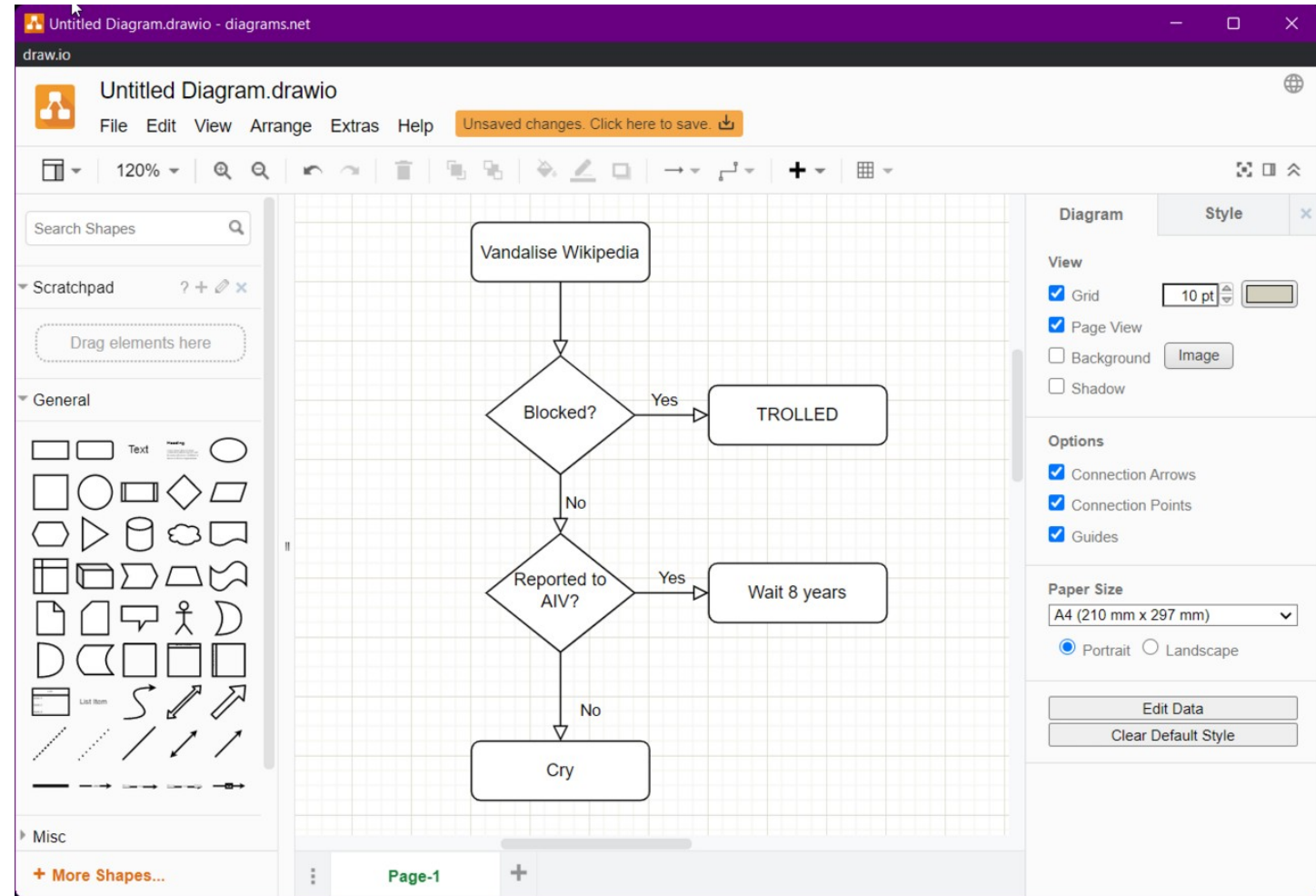
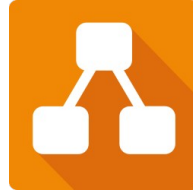
NVIDIA Omnigraph

Actions and Reactions
Nodes and Connections

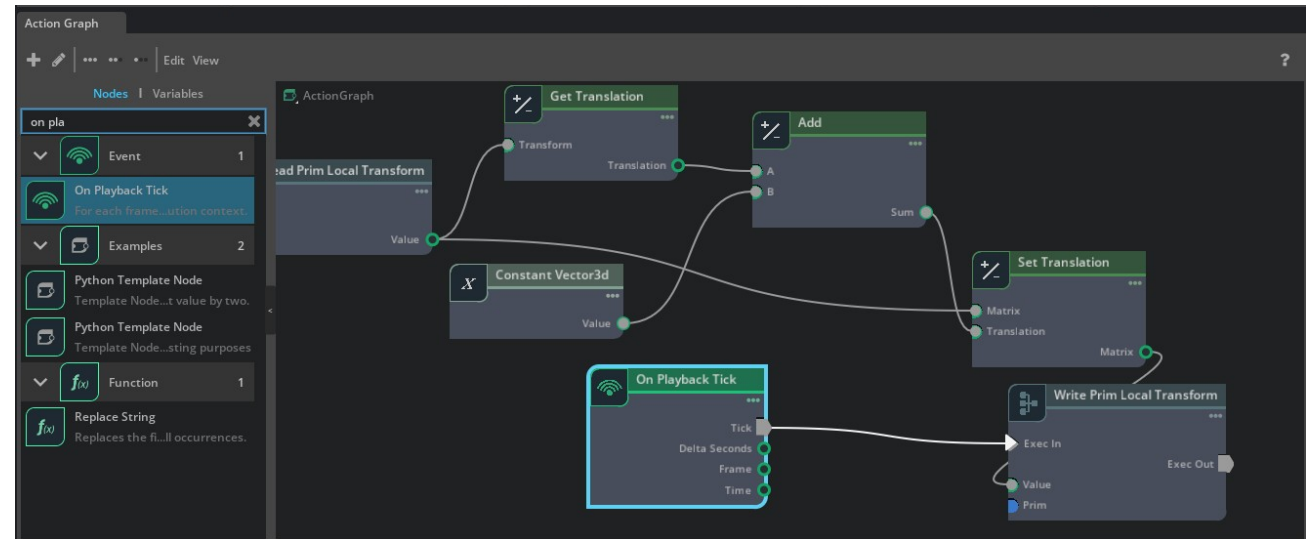
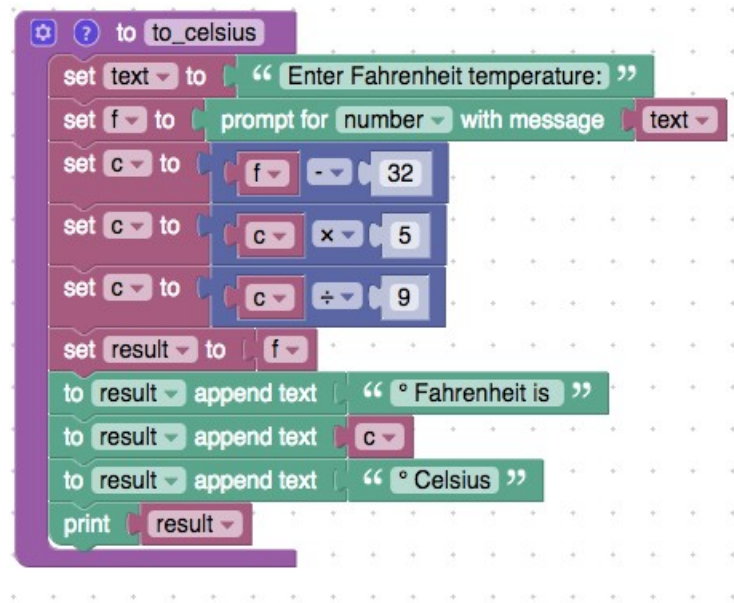


Draw.io

Is this a Visual DSL?



More Examples?



For non-devs

Live programming for non-devs

Potluck: dynamic documents

<https://www.inkandswitch.com/potluck/>

TextAlive: Kinetic typography

<https://junkato.jp/textalive/>

Summary

Summary

Programming eXperience (PX)

Live Programming

How to: Traceability

Visual DSLs

Next Class → Types

